索引

1.概述

- 索引（index）是帮助 MySQL 高效获取数据的数据结构（有序）
- 在数据之外，数据库系统还维护着满足特定查找算法的数据结构，这些数据结构以某种方式引用数据
- 这样就可以在这些数据结构上实现高级查找算法，这种数据结构就是索引

2.优劣

2.1 优势

- 类似于书籍的目录索引，提高数据检索的效率，降低数据库的IO成本
- 通过索引列对数据进行排序，降低数据排序的成本，降低 CPU 的消耗

2.2 劣势

- 索引也是一张表，保存了主键与索引字段，并指向实体类的记录，所以索引列也占用空间
- 查询效率提升的同时也降低更新表的速度，对表数据增删改的时候不仅要保存数据，还要更新索引

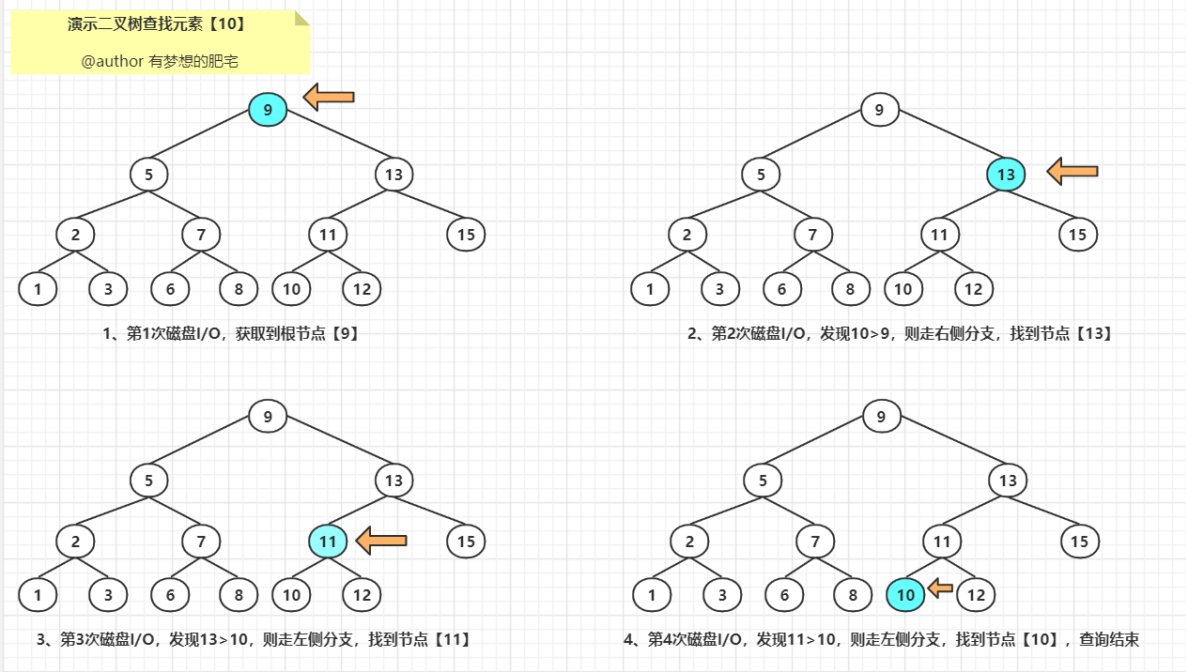
3.结构

3.1 二叉树

- 二叉树即二叉平衡树
- 二叉树特征
  - 非叶子节点都有2个分支(孩子节点)
  - 左侧分支的值比右侧分支的元素要小(左小右大)
  - 查询元素时通过与当前节点元素进行比较大小
    - 若小于当前节点的元素走左侧分支
    - 大于当前节点的元素时，则走右侧分支
- 优势
  - 若非最悲观的情况（即查到最后一层）或者查询的层数不高的时候，可以比较快的返回相应的元素
  - 每个节点都存有对应的元素，若获取的元素正好在当前节点时，则不再继续往下走，磁盘IO就会少一些

- 劣势
  - 当查询深度较高时，会增加磁盘IO的次数，影响效率（图中以每取一次节点进行一次磁盘IO为例）
  - 按顺序插入时不会自平衡，很容易变成“瘸腿树"

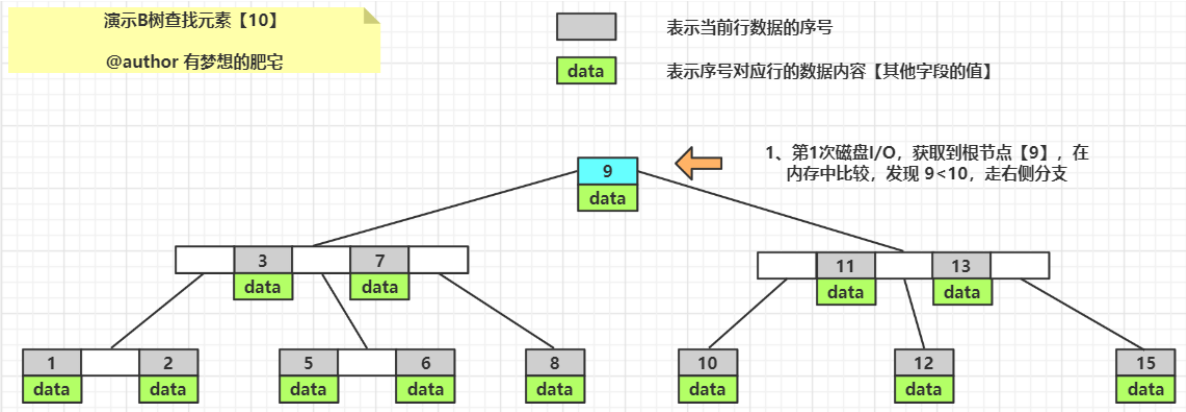
3.1.1 查找元素

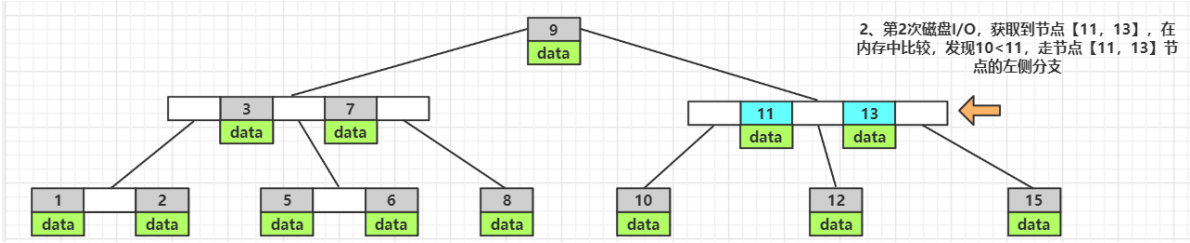


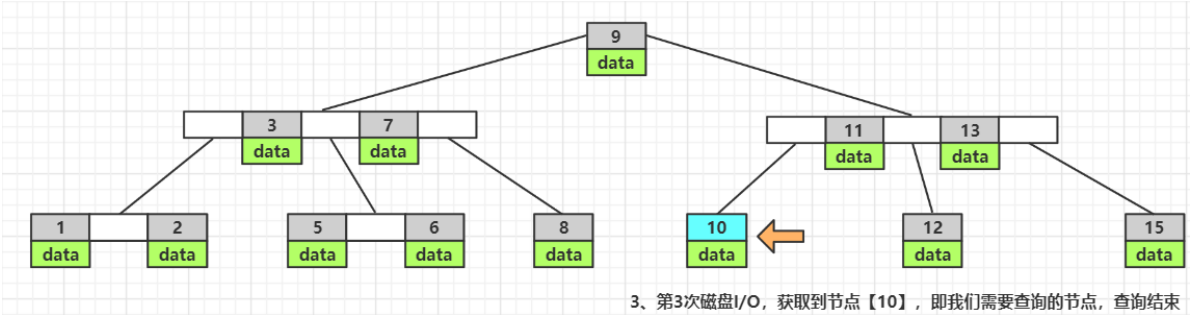
3.2 B树

- B树即多路平衡搜索树
- 特征
  - 树中每个节点最多包含 m 个孩子
  - 除根节点与叶子节点外，每个节点至少有 [ceil(m/2)] 个孩子
  - 若根节点不是叶子节点，则至少有两个孩子
  - 所有的叶子节点都在同一层
  - 每个非叶子节点由 n 个 key 与 n+1 个指针组成，其中 [ceil(m/2)-1] <= n <= m-1

3.2.1 查找元素



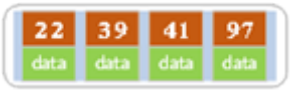




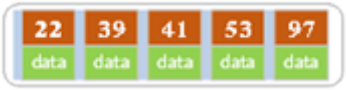
3.2.2 添加元素

- 五阶B树，节点最多4个，最少2个
- 在空树中插入 39，此时根结点就一个key，此时根结点也是叶子结点

- 继续插入22，97和41，根结点此时有4个key



- 继续插入53

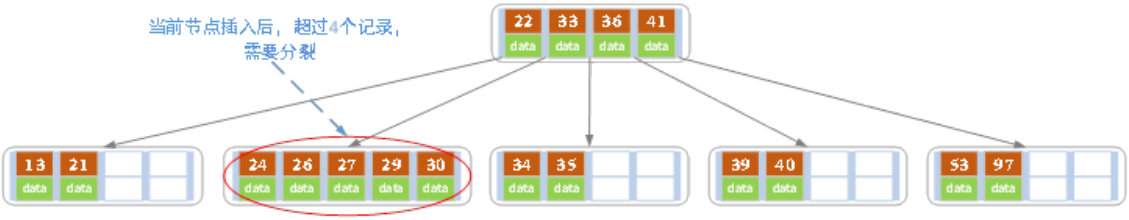


  - 插入后超过了最大允许的关键字个数4，所以key值以41为中心进行分裂
  - 分裂后当前结点指针指向父结点，满足B树条件，插入操作结束
  - 当阶数m为偶数时，需要分裂时就不存在排序恰好在中间的key
  - 那么我们选择中间位置的前一个key或中间位置的后一个key为中心进行分裂即可

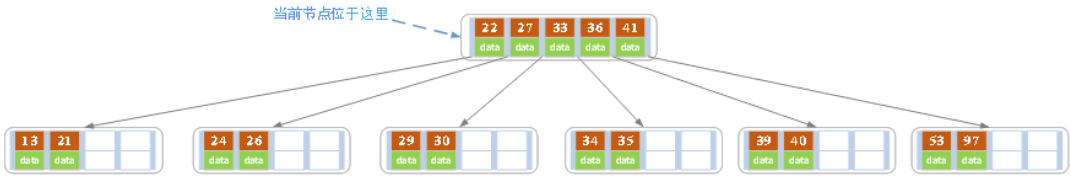
- 依次插入13，21，40，同样会造成分裂

- 依次插入30，27, 33；36，35，34；24，29

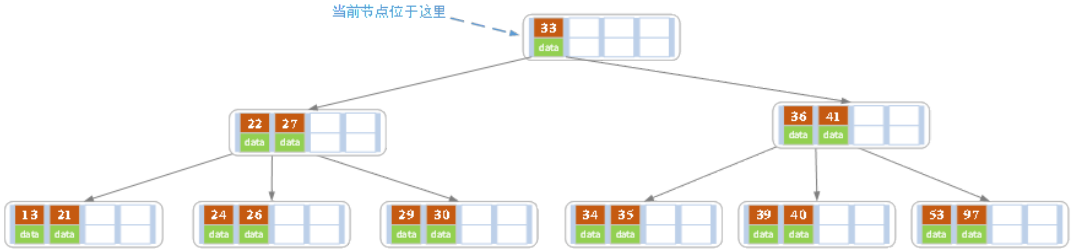
- 插入key值为26的记录



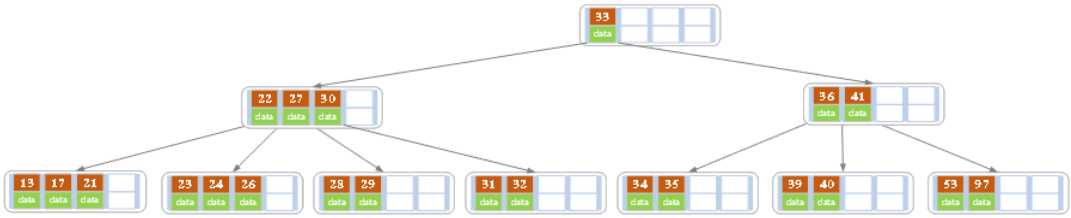
  - 当前结点需要以27为中心分裂，并向父结点进位27，然后当前结点指向父结点



  - 进位后导致当前结点（即根结点）也需要分裂，分裂后当前结点指向新的根，此时无需调整

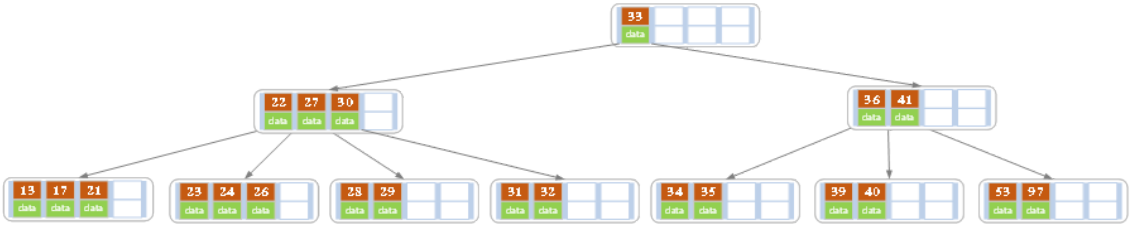


  - 最后再依次插入key为17,28,23,31,32的记录

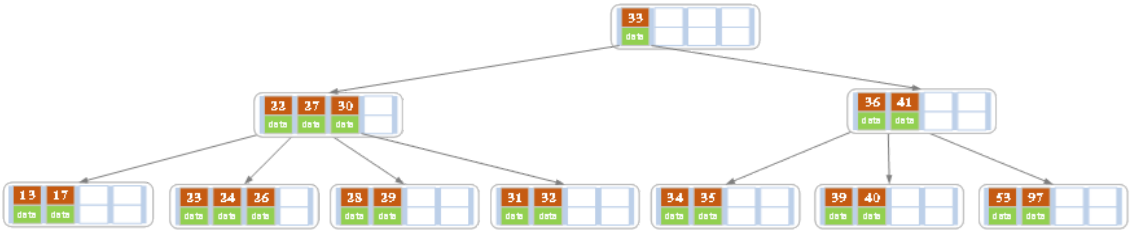


3.2.3 删除元素

- 原始状态

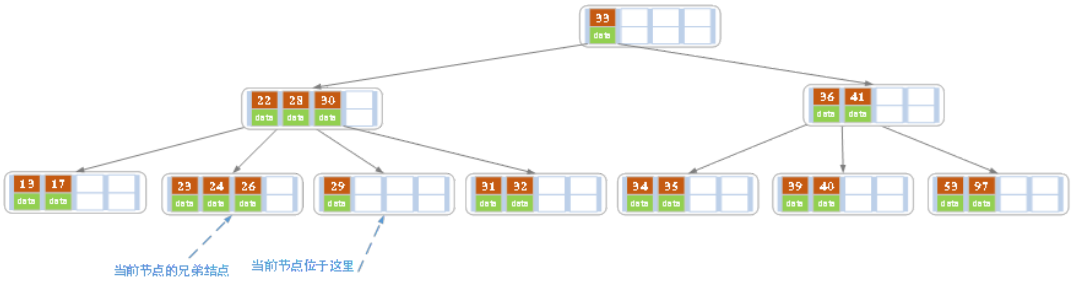


- 在上面的B树中删除21，删除后结点中的关键字个数仍然大于等2，所以删除结束

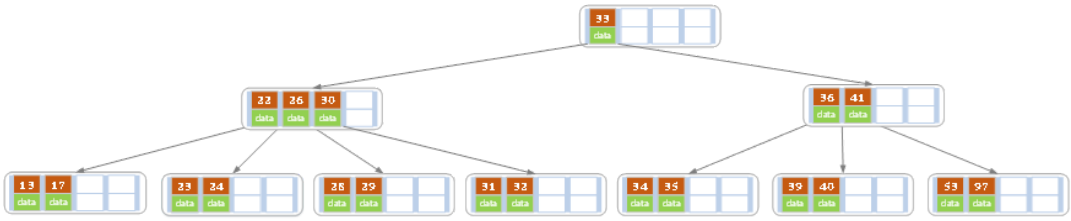


- 在上述情况下接着删除27

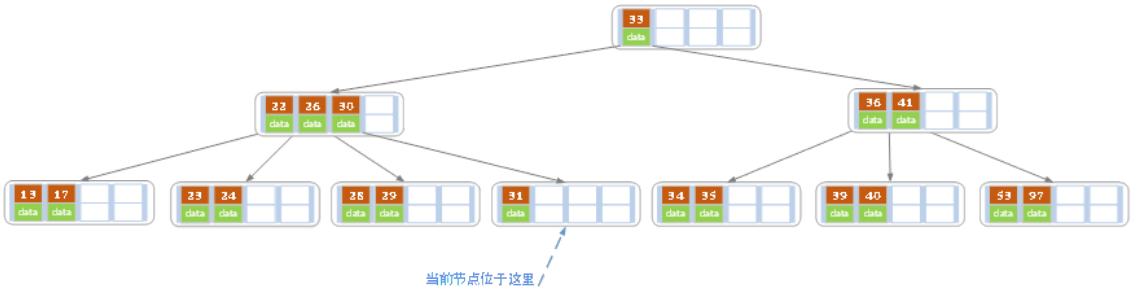
- 从上图可知27位于非叶子结点中，所以用27的后继替换它
- 27的后继为28，我们用28替换27，然后在28（原27）的右孩子结点中删除28



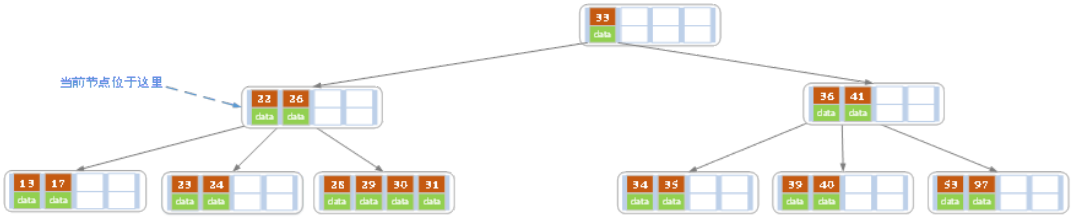
- 删除后发现，当前叶子结点记录数小于2，而兄弟结点有3个记录
- 当前结点还有一个右兄弟，选择右兄弟就会出现合并结点的情况
- 不论选哪一个都行，只是最后B树的形态会不一样而已
- 我们可以从兄弟结点中借取一个key。所以父结点中的28下移，兄弟结点中的26上移,删除结束



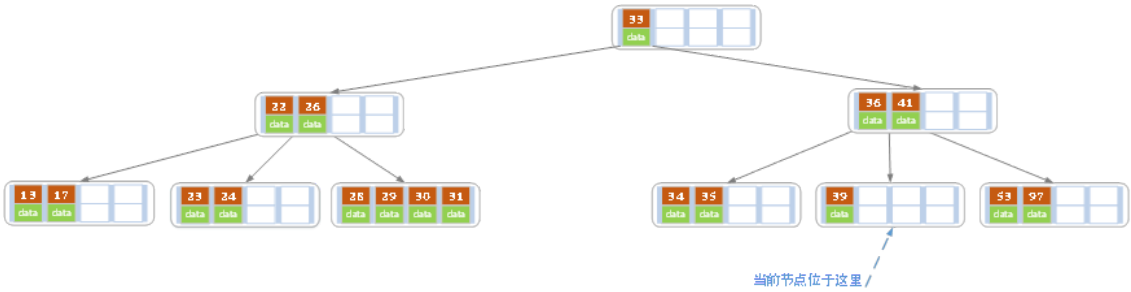
- 在上述情况下接着32



- 当删除后，当前结点中只有一个key，而兄弟结点中也仅有2个key
- 只能让父结点中的30下移和两个孩子结点中的key合并，成为一个新的结点，当前结点的指针指向父结点
- 当前结点key的个数满足条件，删除结束

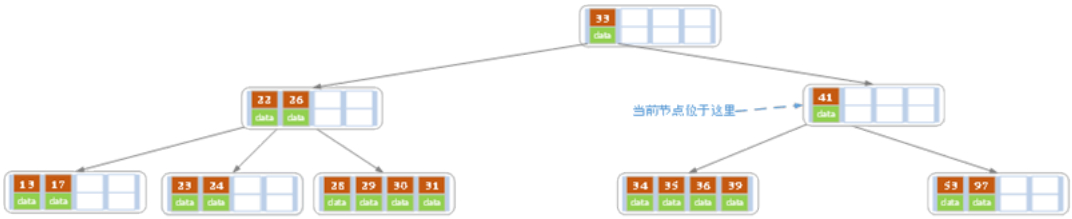


- 上述情况下接着删除key为40的记录

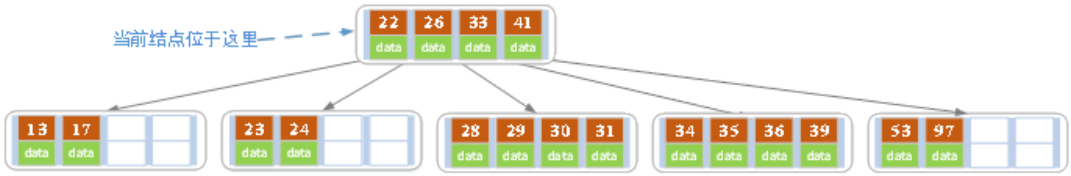


- 同理，当前结点的记录数小于2，兄弟结点中没有多余key

- 父结点中的key下移，和兄弟（这里我们选择左兄弟，选择右兄弟也可以）结点合并
- 合并后的指向当前结点的指针就指向了父结点

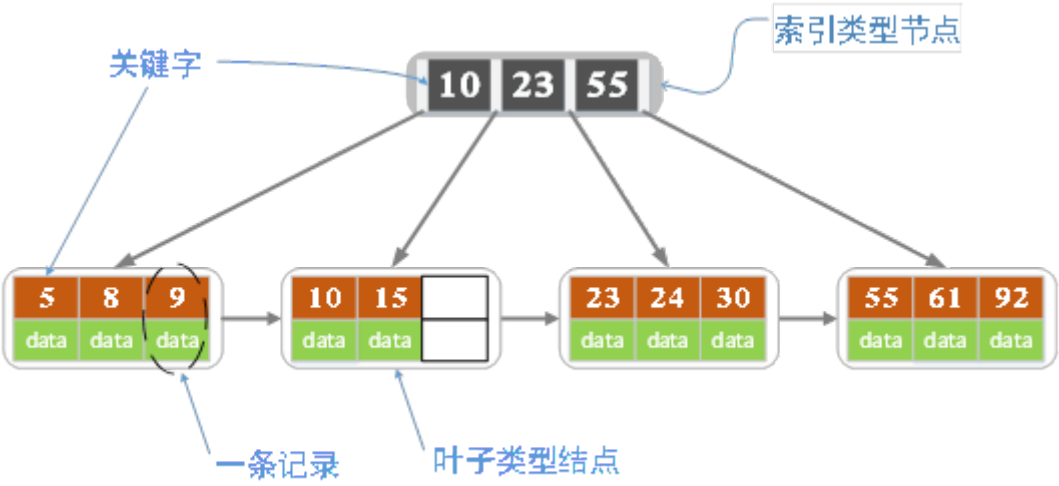


- 同理，对于当前结点而言只能继续合并，合并后结点当前结点满足条件，删除结束



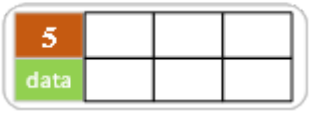
3.3 B+树

- B+树包含2种类型的结点
  - 内部结点（也称索引结点）
    - 本身即可以是内部结点
  - 叶子结点
    - 根结点也可以是叶子结点。
- 根结点的关键字个数最少可以只有1个
- B+树与B树最大的不同是内部结点不保存数据，只用于索引，所有数据（或者说记录）都保存在叶子结点中
- m阶B+树表示了内部结点最多有m-1个关键字（或者说内部结点最多有m个子树）
- 阶数m同时限制了叶子结点最多存储m-1个记录
- 内部结点中的key都按从小到大的顺序排列，左树中的所有key都**小于**它，右子树中的key都**大于等于**它
- 叶子结点中的记录也按照key的大小排列
- 每个叶子结点都存有相邻叶子结点的指针，叶子结点本身依关键字的大小自小而大顺序链接



3.3.1 添加元素

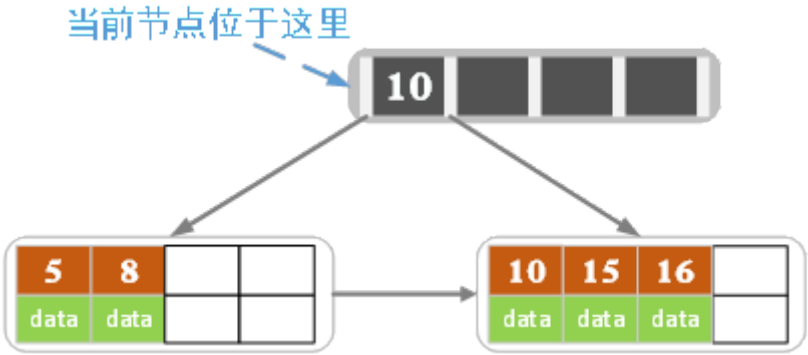
- 5阶B+树的插入过程，5阶B+数的结点最少2个key，最多4个key
- 空树中插入5



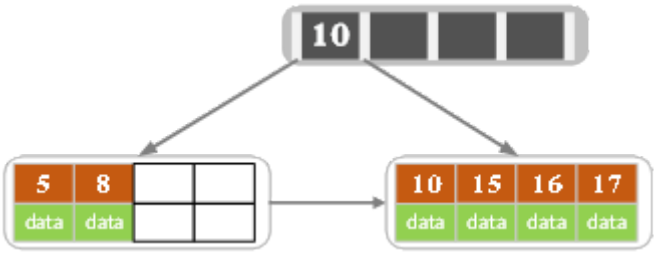
- 依次插入8，10，15

- 插入16

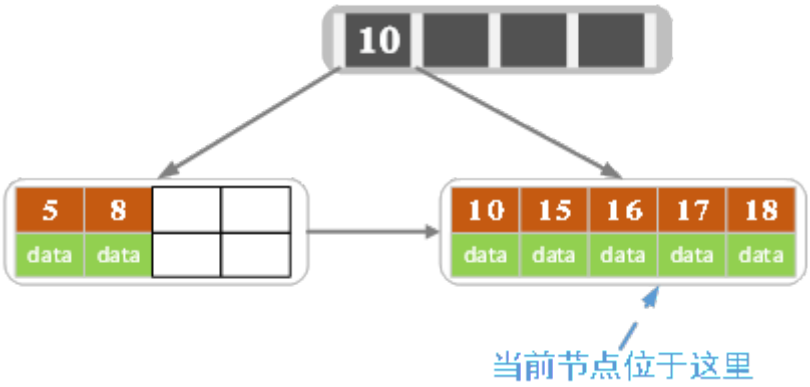
  - 插入16后超过了关键字的个数限制，所以要进行分裂
  - 在叶子结点分裂时，分裂出来的左结点2个记录，右边3个记录或左结点3个记录，右结点2个记录
  - 中间key成为索引结点中的key，分裂后当前结点指向了父结点（根结点）



- 插入17

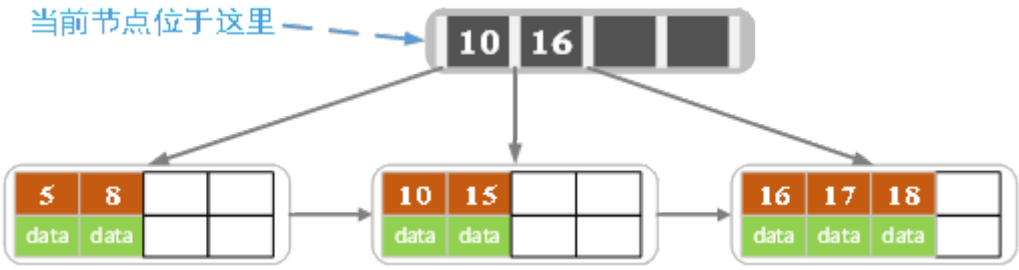


- 插入18

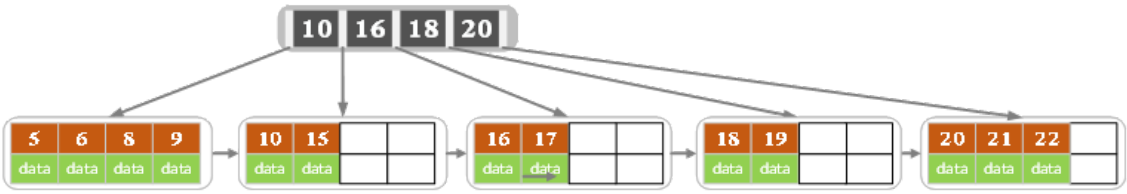


  - 当前结点的关键字个数大于5，进行分裂

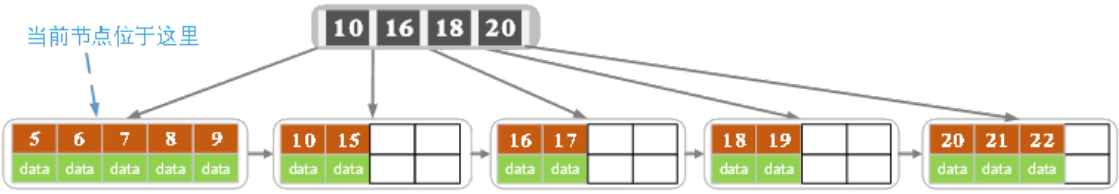
- 分裂成两个结点，左结点2个记录，右结点3个记录
- 关键字16进位到父结点（索引类型）中，将当前结点的指针指向父结点



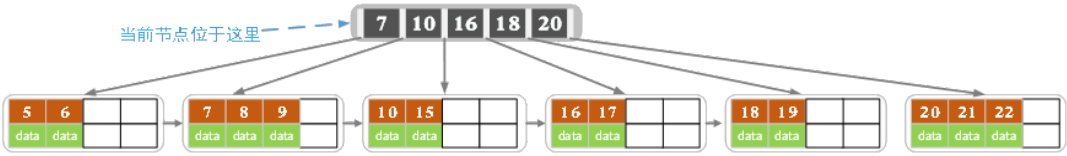
- 插入若干数据后



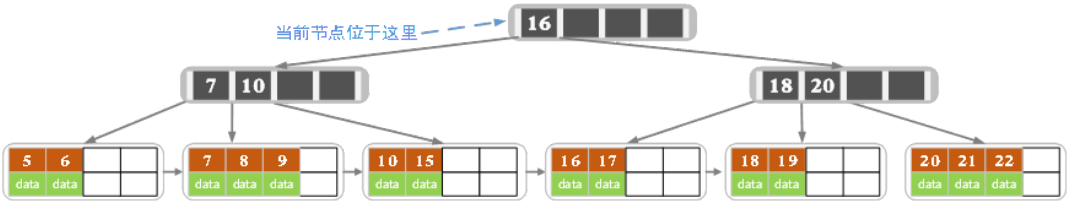
- 插入7



- 当前结点的关键字个数超过4，需要分裂
- 左结点2个记录，右结点3个记录
- 分裂后关键字7进入到父结点中，将当前结点的指针指向父结点

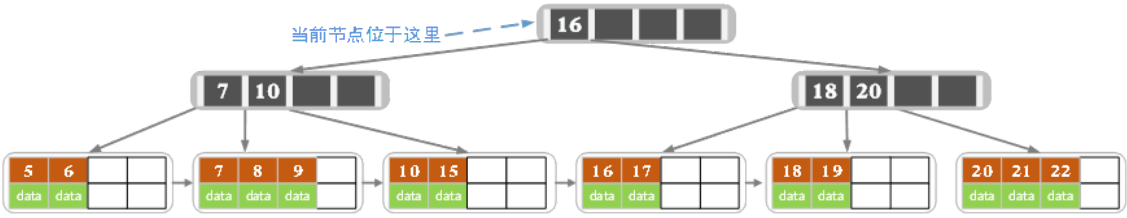


- 当前结点的关键字个数超过4，需要继续分裂
- 左结点2个关键字，右结点2个关键字
- 关键字16进入到父结点中，将当前结点指向父结点

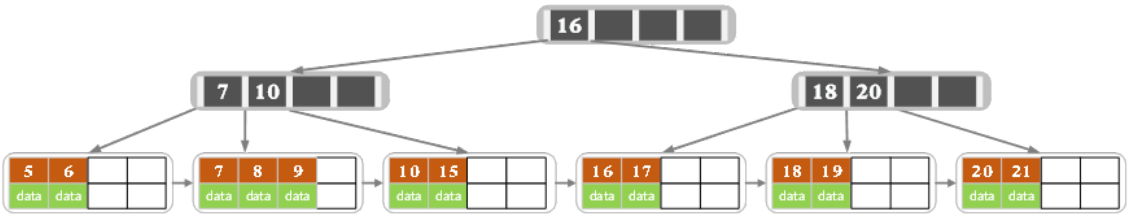


3.3.2 删除元素

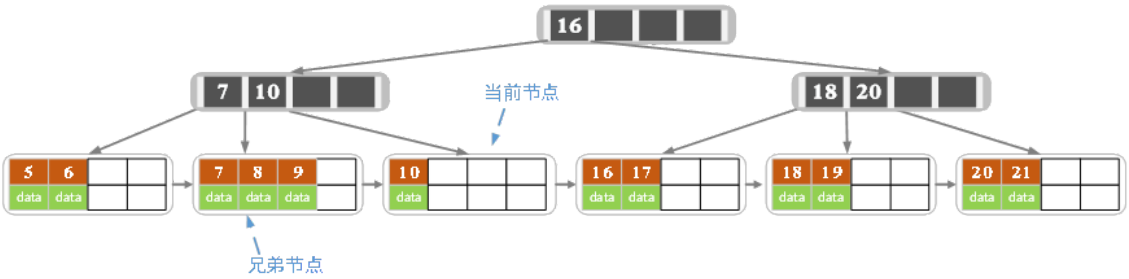
- 初始状态



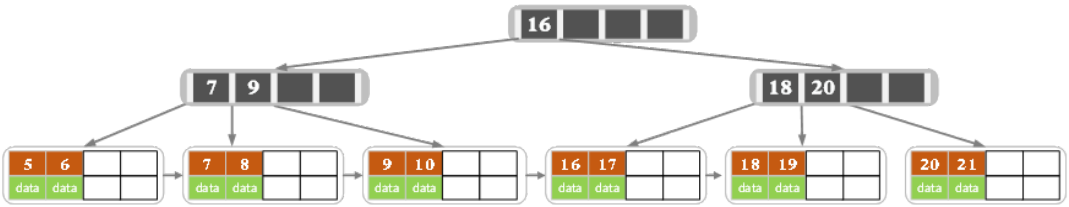
- 删除22，删除后叶子结点中key的个数大于等于2，删除结束



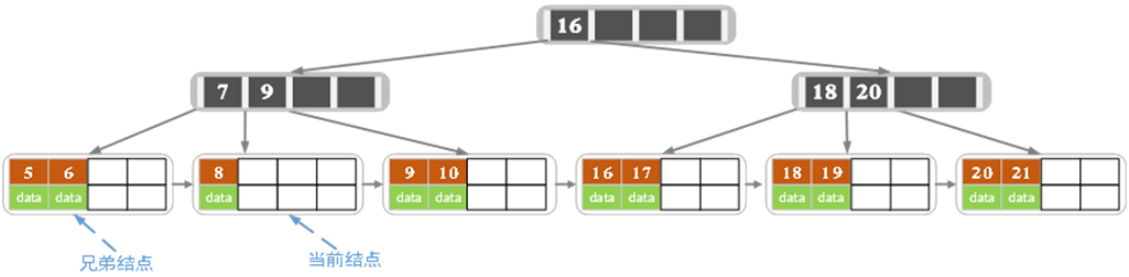
- 删除15



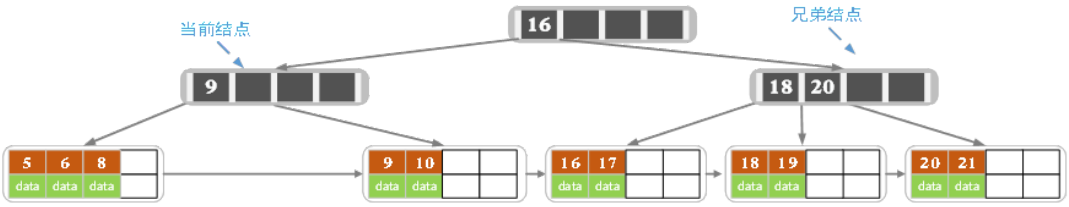
  - 删除后当前结点只有一个key,不满足条件
  - 而兄弟结点有三个key，可以从兄弟结点借一个关键字为9的记录
  - 同时更新将父结点中的关键字由10也变为9，删除结束



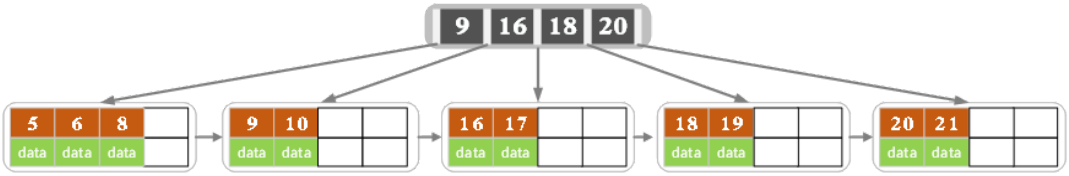
- 删除7



  - 当前结点关键字个数小于2
  - 兄弟结点中的也没有富余的关键字（左右兄弟节点任选其一）
  - 所以当前结点和兄弟结点合并，并删除父结点中的key，当前结点指向父结点



  - 此时当前结点的关键字个数小于2，兄弟结点的关键字也没有富余
  - 所以父结点中的关键字下移，和两个孩子结点合并



4.Explain

4.1 概述

- 我们在数据库中运行 SQL 语句时，底层优化器会选择最优的执行方式来执行我们的 SQL 语句
- 也就是说，底层执行 SQL 语句的顺序并不一定跟我们写的 SQL 语句的顺序是一致的
- Explain 是 Mysql 对 SELECT 的优化器，目的是通过计算分析系统中收集到的统计信息，为 Query(查询) 提供它认为最优的执行计划
- Explain 优化器可以让我们知道数据库对 SQL 语句的执行计划是怎么样的

  比如多表连接的顺序是什么，执行一条 SQL 语句时，数据库在执行的时候先做什么，再做什么
- explain 语句有很多的输出项，我们就可以通过这些输出项的数据来对 SQL 语句做判断，从而优化那些查询慢的 SQL 语句
- 使用 explain 关键字执行一条语句时，它只是输出当前 SQL 语句最优的执行计划，但并没有真正的去执行这条 SQL 语句

4.2 使用

- explain 关键字的使用方式极其简单，我们只需要将它加在SQL 语句的最前面就可以了

```sql
# 加在 SELECT 查询语句的前面
EXPLAIN SELECT * FROM student WHERE id > 115;
# 加在 DELETE 删除语句的前面
EXPLAIN DELETE FROM student WHERE id = 110;
# 加在 UPDATE 更新语句的前面
EXPLAIN UPDATE student SET studentName = "张三" WHERE id = 130;
```

4.3 Explain 各字段含义

- 执行 explain 会输出很多的字段，一共有12个，每个字段代表了不同的含义，列举如下

列名	描述
id	在一个大的查询语句(子查询)中每个select关键字都对应一个唯一的id
select_type	select关键字对应的那个查询的类型
table	表名
partitions	匹配的分区信息
type	针对单表的访问方法
possible_keys	可能用到的索引
key	实际上使用的索引
key_len	实际使用的索引长度
ref	当使用索引列等值查询时，与索引列进行等值匹配的对象信息
rows	预估的需要读取的记录条数
filtered	某个表经过搜索条件过滤后剩余记录条数的百分比
extra	一些额外的信息

- （了解）id 表示查询的一个序列号，用来表示查询中执行select子句或操作表的顺序

    - 相同时，查询的执行顺序为由上至下
    - 不同时，如果存在子查询，id的序号会递增，执行顺序由大到小
- （了解）select_type 查询的类型，用于区别普通查询、联合查询、子查询或其他的复杂查询
    - SIMPLE

      简单的select查询，sql语句中没有使用UNION和子查询
    - PRIMARY

      查询中若包含任何复杂的子部分，最外层查询则被标记为Primary
    - DERIVED

      在FROM列表中包含的子查询，把结果放在临时表里
    - SUBQUERY

      在SELECT或WHERE列表中包含了子查询
    - DEPEDENT SUBQUERY

      在SELECT或WHERE列表中包含了子查询,子查询基于外层
    - UNCACHEABLE SUBQUERY

      无法使用缓存的子查询
    - UNION

      若第二个SELECT出现在UNION之后
    - UNION RESULT

      从UNION表获取结果的SELECT
- （了解）table 数据的来源是那张表
- （熟练掌握）type 查询的访问类型，是较为重要的一个数据参数，该类型可以说直接反映你sql语句执行的快慢

  结果值从最高到最低依次为 **system**>**const**>**eq_ref**>**ref**>fulltext>ref_or_null>index_merge>unique_subquery>index_subquery>**range**>**index**>**all**

  我们平时sql调优的时候，一般得保证查询至少达到range级别，最好能达到ref
    - system ：只有一行记录，这是const类型的特列，开发中可以说不会出现
    - const：通过一次索引就找到，const用于比较 primarykey 或者 unique 索引

      因为只匹配一行数据，所以将主键放到 where 后查询，MySQL就能将该查询转换为一个常量
    - eq_ref：唯一性索引扫描，对于每个索引键，表中只有一条记录与之匹配。常见于主键或唯一索引扫描
    - ref：非唯一性索引扫描，返回匹配某个单独值的所有行
    - range：检索定范围的行，使用一个索引来选择行。一般当你使用where语句中出现了between、<、>、in等的查询时出现
    - index：sql使用了索引但是没用通过索引进行过滤，一般是使用了覆盖索引或者是利用索引进行了排序分组
    - all：全表扫描，遍历全表来找到匹配的行
- （了解）possible_keys 在表中可能使用到的索引，查询涉及到的字段上若存在索引，则该索引将被列出，但不一定被查询实际使用
- （了解）key 真实使用的索引。如果为NULL，表示没有使用索引
- （重点掌握）key_len 索引中使用的字节数。 此字段能够帮检查是否充分利用了索引。ken_len越长说明索引使用得越充分

- （了解）ref 表示索引的那一列被使用了，如果可以的话，最好是一个常量。表示那些列或者常量被用于查找列上的值
- （掌握）rows 显示 MySQL 认为它执行查询时必须检查的行数。 越少越好
- （重要指标） extra 其他的额外重要的信息
  - Using filesort（降低性能）：MySQL 中无法利用索引完成的排序操作称为“文件排序”
  - Using temporary（降低性能）:使用了临时表保存中间结果，MySQL 在对查询结果排序时使用临时表。 常见于排序 order by 和分组查询 group by
  - Using index（提高性能）表示相应的 select 操作中使用了覆盖索引(Covering Index)， 避免访问了表的数据行， 效率不错
    - 如果同时出现 using where， 表明索引被用来执行索引键值的查找
    - 如果没有同时出现 using where， 表明索引只是用来读取数据而非利用索引执行查找。利用索引进行了排序或分组
  - Using where 表明使用了 where 过滤
  - Using join buffer 使用了连接缓存
  - impossible where where 子句的值总是 false， 不能用来获取任何元组

5.使用

5.1 单值索引

- 一个索引只包含单个列，一个表可以有多个单列索引

```
#直接创建
create index 索引名 on 表名(字段名);
#修改表形式创建
alter table 表名 add index 索引名称(字段列表);
```

- 使用单值索引

```
-- 查询所有数据
explain select * from customers
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	customers	(Null)	ALL	(Null)	(Null)	(Null)	(Null)	3	100.00	(Null)

```
-- 按主键为条件过滤
explain select * from customers where id=1
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	customers	(Null)	const	PRIMARY	PRIMARY	4	const	1	100.00	(Null)

```
-- 按非主键为条件过滤
explain select * from customers where name='Ramesh'
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	customers	(Null)	ALL	(Null)	(Null)	(Null)	(Null)	3	33.33	Using wl

```
-- 对非主键字段创建单值索引
create index cus_name_idx on customers(name);
explain select * from customers where name='Ramesh'
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	customers	(Null)	ref	cus_name_idx	cus_name_idx	63	const	1	100.00	(Null)

```
-- 查看当前表的所有索引信息
show index from customers
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment
customers	0	PRIMARY	1	id	A	3	(Null)	(Null)		BTREE		
customers	1	cus_name_idx	1	name	A	3	(Null)	(Null)	YES	BTREE		

5.2 唯一索引

- 索引列的值必须唯一，但允许有空值。相当于UNIQUE约束

```
#直接创建
create unique index 索引名 on 表名(字段名);
#修改表形式创建
alter table 表名 add unique [index] 索引名称(字段列表);
```

- 使用唯一索引

```
-- 删除原索引
drop index cus_name_idx on customers
-- 创建非主键字段的唯一索引
create unique index cus_name_idx_unique on customers(name);
explain select * from customers where name='Ramesh'
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	customers	(Null)	const	cus_name_idx	cus_name_idx	63	const	1	100.00	(Null)

5.3 复合索引

- 一个索引包含多个列
- 比如对 A B C 三个字段创建复合索引
  - 则对A创建索引
  - 对A，B创建索引
  - 对A，B，C创建索引

```
create index 索引名 on 表名(字段名,字段名,...);
```

- 使用复合索引

```
-- 删除原索引
drop index cus_name_idx_unique on customers
create index cus_name_age_address_idx on customers(name,age,address);
explain select * from customers where name='Ramesh'
explain select * from customers where name='Ramesh' and age=32
explain select * from customers where name='Ramesh' and age=32 and
address='Ahmedabad'
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	customers	(Null)	ref	cus_name_age_a	cus_name_age_address	63	const	1	100.00	(Null)

```
explain select * from customers where age=32
explain select * from customers where address='Ahmedabad'
explain select * from customers where name='Ramesh' or salary=2000
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	customers	(Null)	ALL	(Null)	(Null)	(Null)	(Null)	3	33.33	Using wl

```
explain select * from customers where name like 'Rame%'
```

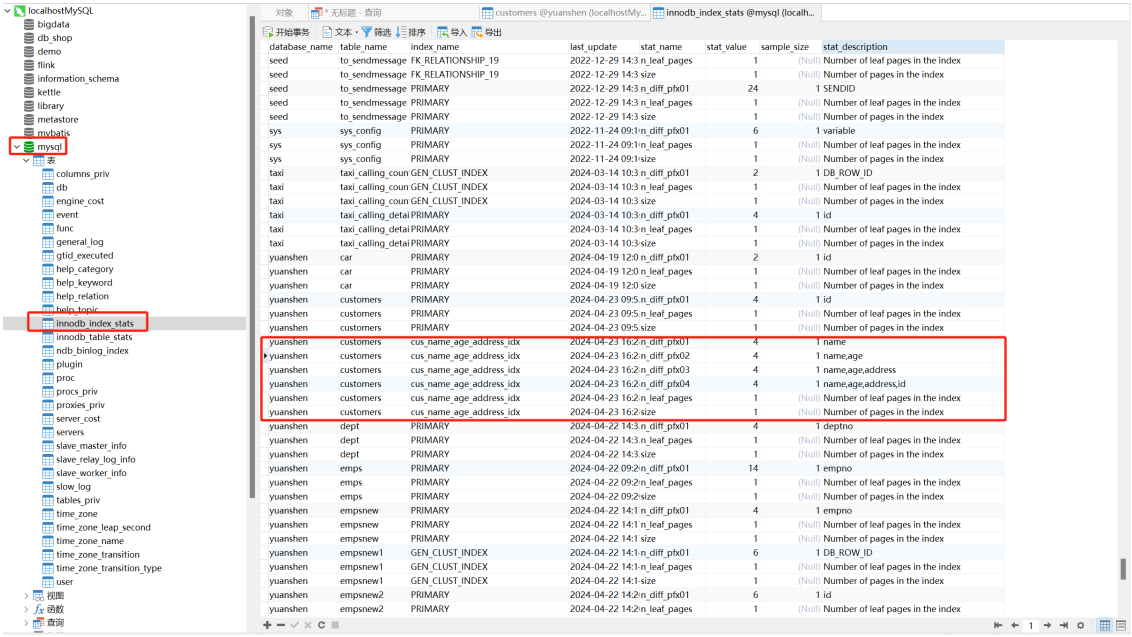
id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	customers	(Null)	range	cus_name_age_a	cus_name_age_address	63	(Null)	1	100.00	Using in

```
explain select * from customers where name like '%Rame%'
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	customers	(Null)	ALL	(Null)	(Null)	(Null)	(Null)	3	33.33	Using wl

5.4 存储

- 所有数据库的所有表的所有索引都存在于系统数据库mysql 的 innodb_index_stats 表中



6.总结

- 对于创建的多列索引，只要查询条件使用了最左边的列，索引一般就会被使用

  例如在a,b,c列上创建索引，那么查询条件"a=5","a=5 and b=6","a=7 and b=8 and c=6"都会使用该索引
- 对于使用like的查询，查询如果是‘%aaa’不会使用到索引‘ aaa%’会使用到索引
- 如果条件中有or，那么除非or条件都带有索引，否则还是会全表扫描
- 连接查询时，在连接键上增加索引可以加快速度，同时遵循小表驱动大表原则
- 如果mysql估计使用全表扫描要比使用索引快，则不使用索引